CALIDAD DEL SOFTWARE

GUION DE PRÁCTICAS 1

Entorno de Calidad, Línea Base y Documentación Técnica

1. **Duración estimada:** 3 sesiones (6 horas).
2. **Herramientas:** Visual Studio Code, Doxygen, Graphviz, SonarQube for IDE.

1. INTRODUCCIÓN Y OBJETIVOS

El objetivo de esta práctica es simular un escenario profesional de **Mantenimiento de Software**. En lugar de comenzar un desarrollo desde cero (*Greenfield*), trabajaremos sobre un proyecto existente (*Brownfield*) para auditar su calidad y prepararlo para futuras mejoras.

Los objetivos específicos son:

1. Configurar un entorno de desarrollo (IDE) preparado para la inspección de calidad estricta.
2. Seleccionar un proyecto en C++ con deuda técnica (complejidad alta o métodos excesivamente largos).
3. Utilizar herramientas de análisis estático (**SonarQube for IDE**) para identificar puntos críticos.
4. Generar documentación técnica profesional incluyendo diagramas automáticos (**Doxygen + Graphviz**).

2. PREPARACIÓN DEL ENTORNO DE TRABAJO

Para asegurar la reproducibilidad de los resultados entre alumnos y profesor, es necesario instalar el software siguiendo estrictamente estos pasos y versiones.

2.1. Software Base (Instalación en S.O.)

1. **Visual Studio Code (VS Code):**
   - Accede a code.visualstudio.com.
   - Haz clic en el botón de descarga ("Download for Windows") e instálalo aceptando las opciones por defecto.
2. **Compilador de C++ (MinGW-w64 - GCC 15.2.0):**
   - **Descarga:** Web de winlibs.com. Busca la sección **UCRT runtime** dentro de **Release Versions** y descarga el **ZIP** de la última versión disponible (**64-bit**).
   - **Instalación:** Descomprime en **C:\mingw64**.
   - **PATH:** Añade C:\mingw64\bin a las Variables de Entorno del sistema.
   - **Verificación:** Desde **cmd** de Windows > g++ --version.
3. **Motor Java (IMPRESCINDIBLE para SonarQube):**
   - **Descarga:** Eclipse Adoptium Temurin JDK 21 (LTS).
   - ⚠️ **MUY IMPORTANTE:** Marca la opción **"Set JAVA_HOME variable"** durante la instalación.
   - **Verificación:** Desde **cmd** de Windows > java -version
4. **Herramientas de Documentación:**
   - **Doxygen:** Descargar e instalar desde doxygen.nl.
     - *Nota:* Habitualmente se instala en C:\Program Files\doxygen\bin.
   - **Graphviz:** Descargar el instalador (EXE) desde graphviz.org.
     - Selecciona la opción *"Add Graphviz to the system PATH for all users"*.
     - Habitualmente se instala en C:\Program Files\Graphviz\bin.
     - **Verificación**: Desde **cmd** de Windows > dot -V

2.2. Configuración de Visual Studio Code (Extensiones)

Instala las siguientes extensiones verificando su ID:

1. **C/C++ Extension Pack** (Microsoft) - ID: ms-vscode.cpptools-extension-pack
   - Esta extensión instalará automáticamente las extensiones C/C++ y CMake Tools
2. **SonarQube for IDE** (SonarSource) - ID: SonarSource.sonarlint-vscode
3. **Doxygen Documentation Generator** (Christoph Schmalhofer) - ID: cschlosser.doxdocgen

2.3. SonarQube: Solución de Problemas Frecuentes

Es posible que al empezar a trabajar, SonarQube muestre alertas o no se comporte como esperas. Aquí tienes cómo proceder:

2.3.1. Aviso: "Unable to locate compilation database"

Síntoma: Aparece un mensaje indicando que no se encuentra la base de datos de compilación.

Solución: Haz clic en **"Don't show again"**. Más adelante crearemos el fichero compile_commands.json y el problema quedará resuelto.

2.3.2. Error: "Node.js runtime version... is required"

Síntoma: En la pestaña Output aparece un error diciendo que se requiere Node.js 20.x o superior.

Causa: La extensión se ha actualizado a una versión del fichero node.js más moderna que la incluida en la actual versión de Visual Studio Code.

Solución: Ignorar el mensaje (SonarQube seguirá funcionando correctamente) o restaurar una versión anterior de SonarQube (si no quieres seguir viendo el mensaje... 😅):

1. Ve a la pestaña de **Extensiones**.
2. Selecciona **SonarQube**.
3. Haz clic en la **flecha hacia abajo (v)** junto al botón **Uninstall**.
4. Selecciona **"Install Specific Version..."** .
5. Elige una versión más antigua (de hace unos meses).
6. Reinicia VS Code.

2.4. Configuración Global en Visual Studio Code

Configura los umbrales de calidad en **Usuario** (**F1** o Ctrl+Shift+P > **Preferences: Open User Settings (JSON)**). Asegúrate de incluir las llaves { }:

```JSON
{
    "sonarlint.rules": {
        // S1541: Complejidad Ciclomática (máximo 10 caminos)
        "cpp:S1541": {
            "level": "on",
            "parameters": {
                "maximumFunctionComplexityThreshold": "10"
            }
        },
        // S138: Longitud de función (máximo 25 líneas)
        "cpp:S138": {
            "level": "on",
            "parameters": {
                "max": "25"
            }
        }
    },

    // --- CONFIGURACIÓN DE DOCUMENTACIÓN (Doxygen) ---
    "doxdocgen.c.firstLine": "/**",
    "doxdocgen.c.commentPrefix": " * ",
    "doxdocgen.c.lastLine": " */",
    "doxdocgen.c.triggerSequence": "/**"
}
```

3. SELECCIÓN Y PREPARACIÓN DEL PROYECTO

3.1. Estructura de Carpetas

1. Crea la carpeta de tu práctica.
    - **Importante**: Evita las rutas con tildes u otros símbolos extraños.
2. Copia dentro los archivos fuente (.cpp, .h).
3. Crea una subcarpeta llamada .vscode

3.2. Configuración Automática (.vscode)

Crea estos tres archivos dentro de la carpeta .vscode:

Archivo 1: tasks.json

```JSON
{
    "version": "2.0.0",
    "tasks": [
        {
            "type": "cppbuild",
            "label": "Compilar Activo (G++)",
            "command": "C:/mingw64/bin/g++.exe",
            "args": [
                "-fdiagnostics-color=always", "-g", "${workspaceFolder}/*.cpp",
                "-o", "${workspaceFolder}/main.exe",
                "-std=c++17", "-Wall", "-Wextra"
            ],
            "options": { "cwd": "${workspaceFolder}" },
            "problemMatcher": ["$gcc"],
            "group": { "kind": "build", "isDefault": true }
        }
    ]
}
```

Archivo 2: launch.json

```JSON
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "Ejecutar Programa",
            "type": "cppdbg",
            "request": "launch",
            "program": "${workspaceFolder}/main.exe",
            "args": [],
            "stopAtEntry": false,
            "cwd": "${workspaceFolder}",
            "environment": [],
            "externalConsole": true,
            "MIMode": "gdb",
            "miDebuggerPath": "C:/mingw64/bin/gdb.exe",
            "setupCommands": [
                { "text": "-enable-pretty-printing", "ignoreFailures": true }
            ],
            "preLaunchTask": "Compilar Activo (G++)"
        }
    ]
}
```

Nota: Cambia **externalConsole** a **false** si prefieres utilizar el terminal de VS Code.

Archivo 3: c_cpp_properties.json

```JSON
{
    "configurations": [
        {
            "name": "Win32",
            "includePath": [
                "${workspaceFolder}/**"
            ],
            "defines": [
                "_DEBUG",
                "UNICODE",
                "_UNICODE"
            ],
            "compilerPath": "C:/mingw64/bin/g++.exe",
            "cStandard": "c17",
            "cppStandard": "gnu++17",
            "intelliSenseMode": "windows-gcc-x64",
            "compileCommands": [
                "${workspaceFolder}/compile_commands.json"
            ]
        }
    ],
    "version": 4
}
```

3.3. SonarQube: Configuración del Enlace de Compilación

1. Crea el archivo compile_commands.json en la carpeta raíz de tu proyecto (no dentro de .vscode).
2. Copia el siguiente contenido dentro del fichero:

```JSON
[
  {
    "directory": "C:/RUTA/ABSOLUTA/DE/TU/CARPETA",
    "command": "g++ -c main.cpp",
    "file": "main.cpp"
  },
  {
    "directory": "C:/RUTA/ABSOLUTA/DE/TU/CARPETA",
    "command": "g++ -c archivo1.cpp",
    "file": "archivo1.cpp"
  },
  {
    "directory": "C:/RUTA/ABSOLUTA/DE/TU/CARPETA",
    "command": "g++ -c archivo2.cpp",
    "file": "archivo2.cpp"
  }
]
```

3. Sustituye la ruta incluida en **"directory"** por la ruta real de tu proyecto. Fíjate en que debes usar la **barra normal** ("/") y no la barra invertida.
4. Sustituye archivo1.cpp, archivo2.cpp, etc. por el nombre real de cada uno de los ficheros .cpp que tengas en tu proyecto.
5. En caso de que tengas que añadir nuevos bloques de compilación al fichero compile_commands.json, no olvides utilizar el símbolo "," (coma) para separarlos.

3.4. SonarQube: Dónde Encontrar las Instrucciones de Compilación

Vamos a indicarle ahora a SonarQube dónde encontrar el fichero compile_commands.json que acabamos de crear:

1. Crea el archivo settings.json dentro de la carpeta .vscode/
2. Añade el siguiente contenido:

```JSON
{

"sonarlint.pathToCompileCommands":"${workspaceFolder}/compile_commands.json"
}
```

3.5. Resumen: ¿Para qué sirve cada archivo JSON?

Tras configurar el entorno, habrás creado o modificado varios archivos. Aquí tienes una guía rápida para entender qué controla cada uno:

Ubicación	Archivo	¿Cuál es su función?
Global (Usuario)	**User Settings**	**Reglas generales:** Define los umbrales de SonarQube (complejidad y longitud) y el formato de Doxygen para todos tus proyectos.
Raíz del Proyecto	**compile_commands.json**	**Mapa de Compilación:** Es la guía que usa SonarQube para "leer" y analizar tu código .cpp correctamente.
Carpeta .vscode/	**settings.json**	**Vínculo de Proyecto:** Conecta VS Code con el mapa de compilación anterior.
Carpeta .vscode/	**c_cpp_properties.json**	**IntelliSense:** Gestiona el autocompletado y evita subrayados de error falsos en los #include.
Carpeta .vscode/	**tasks.json**	**Compilador:** Contiene la instrucción exacta para convertir tus archivos .cpp en un ejecutable .exe.
Carpeta .vscode/	**launch.json**	**Ejecución:** Controla cómo se lanza el programa y activa el depurador para buscar fallos.

💡 **Concepto: Configuración Global vs. Local**

Es fundamental distinguir entre estos dos niveles de configuración:

🚀 **Configuración Global (User Settings)**: Son tus preferencias como usuario. Se guardan en tu instalación de VS Code y se aplican a cualquier código que abras en este ordenador.

💻 **Configuración Local (Archivos del Proyecto)**: Son las instrucciones específicas para que este proyecto concreto funcione. Al estar guardados dentro de la carpeta del proyecto (.vscode o raíz), aseguran que cualquier persona que abra tu carpeta (como el profesor) trabaje exactamente en el mismo entorno que tú.

4. DESARROLLO (Línea Base y Validación)

En esta fase debes determinar si tu proyecto es válido para la asignatura y asegurarte de que funciona correctamente.

4.1. Auditoría de Validez (SonarQube)

Para que un proyecto sea válido para las prácticas, debe tener suficiente "deuda técnica". No necesitas entender ahora qué significan estas métricas en profundidad (lo haremos en la Práctica 4), pero sí necesitas verificar que tu código las “incumple”.

Procedimiento de Validación:

1. Abre los archivos .cpp de tu proyecto uno por uno.
2. Espera a que SonarQube analice el código (aparecerán subrayados ondulados).
3. Abre la pestaña **"PROBLEMAS"** (parte inferior) y utiliza el filtro para buscar lo siguiente:
    - **Prioridad 1:** Busca la regla **S1541** (Complejidad Ciclomática).
        - Si aparecen avisos de este tipo, anota en tu memoria: *Nombre del archivo, nombre de la función y el valor actual de complejidad* (ej: "Complejidad 15, permitido 10").
        - **Si encuentras esto, tu proyecto es VÁLIDO.**
    - **Prioridad 2 (Solo si no hay S1541):** Busca la regla **S138** (Longitud de función).
        - Si tu proyecto no tiene problemas de complejidad, verifica si tiene funciones de más de 25 líneas.
        - Si aparecen, anótalas. Tu proyecto es válido por longitud.
    - **Caso Nulo:** Si tras revisar todos los archivos .cpp no aparece ningún aviso de S1541 ni S138, **tu proyecto NO es válido** (es demasiado simple o está demasiado limpio). Deberás buscar otro proyecto propio más complejo o solicitar uno al profesor.

NOTA: En esta práctica **NO tienes que corregir** los avisos de SonarQube. El objetivo ahora es solo detectarlos y documentarlos para establecer la "Línea Base".

4.2. Compilación y Ejecución (El Compilador)

Una vez validado que el proyecto tiene la complejidad adecuada, debes asegurar que funciona:

1. Abre el archivo main.cpp.
2. Pulsa **F5**.

Gestión Inteligente de Errores (Truco !sonar)

Al pulsar F5, la pestaña **Problemas** mostrará mezclados los errores de compilación y los avisos de SonarQube. Para evitar confusiones, sigue estos pasos:

1. **Ignora la Terminal:** Es difícil de leer. Céntrate solo en la pestaña **Problemas**.
2. **Vista de Árbol:** Haz clic en el icono "View as Tree" (carpeta) a la derecha de la barra de filtro. Esto agrupa los errores por archivo, lo cual es mucho más claro.
3. **EL TRUCO CLAVE:** En la barra de búsqueda ("Filter"), escribe exactamente: **!sonar**
   - Al hacerlo, VS Code **ocultará** los avisos de SonarQube de la pestaña **Problemas**.
   - Solo tendrás que centrarte en los **Errores (Rojos)** y **Warnings (Amarillos)** que provienen del compilador (GCC).
   - *¡Así sabrás exactamente qué tienes que arreglar para que compile!*

Escenario A: Errores de Compilación (Rojo)

1. Son errores de sintaxis (falta un punto y coma, librería no encontrada, etc.).
2. Documenta en la memoria el error encontrado (captura de pantalla) y cómo lo has solucionado.

Escenario B: Warnings del Compilador (Amarillo)

El programa posiblemente funcione, pero es recomendable arreglarlos:

1. Intenta solucionar los *warnings* del compilador siempre que sea factible.
2. Documenta las soluciones adoptadas (incluir capturas de pantalla):
   - Corregido: explicar las modificaciones aplicadas.
   - **No** corregido: justificar la decisión.

5. DOCUMENTACIÓN DEL CÓDIGO (Doxygen)

5.1. Alcance de la Documentación

Debes documentar los siguientes elementos del código:

- Ficheros (descripción de éstos al principio de cada uno de ellos).
- Clases y Structs.
- **Todos los métodos:** Incluidos Constructores, Destructores, *Getters* y *Setters*.
- Variables miembro (atributos).
- **Funciones sueltas en .cpp:** Si tienes funciones (como el main o auxiliares) que están implementadas directamente en el archivo .cpp y no tienen cabecera .h, **también deben documentarse** justo encima de su implementación.

5.2. ¿Dónde documentar?

- **Métodos con .h:** Documentar en el archivo de cabecera (.h).
- **Métodos sin .h:** Documentar en el archivo de implementación (.cpp).

5.3. Generación Automática

Sitúate en la línea superior a la función o clase a documentar. En el caso de la documentación de ficheros, sitúate justo en la primera línea del mismo. Una vez situado el cursor, escribe /** y pulsa **Enter.** VS Code generará la plantilla.

5.4. Ejemplos de Formato

1. Cabecera del Archivo (Al inicio de cada archivo .h o .cpp):

```C/C++
/**
 * @file game_designer.cpp
 * @author your name (you@domain.com)
 * @brief Description of the GameDesigner class implementation.
 * @version 0.1
 * @date 2026-02-12
 *
 * @copyright Copyright (c) 2026
 */
```

2. Clases (Justo antes de la declaración de la clase):

```
C/C++
/**
 * @class Universe
 * @brief Clase principal que gestiona la simulación del universo.
 * Se encarga de controlar el ciclo de vida de los objetos Thanos y Avengers,
 * así como de gestionar la memoria dinámica asociada.
 */
class Universe { ... };
```

3. Métodos (Justo antes de la declaración de la función):

```
C/C++
/**
 * @brief Calcula el impacto del chasquido en la población.
 * Utiliza un algoritmo de reducción logarítmica basado en el poder actual.
 * @param population Cantidad total de habitantes actuales (debe ser > 0).
 * @param power Nivel de energía de las gemas (rango 0.0 a 1.0).
 * @return float Número estimado de habitantes supervivientes.
 */
float calculateSnap(int population, float power);
```

IMPORTANTE: Si la función es void, **NO** debes incluir la etiqueta `@return`.

4. Variables Miembro (Al final de la línea):

```
C/C++
private:
    int id;           ///< Identificador único del objeto en la base de datos.
    float energy;     ///< Nivel de energía actual [0-100].
```

6. GENERACIÓN DEL INFORME TÉCNICO (Doxywizard)

Para generar la web de documentación, usaremos la herramienta gráfica **Doxywizard**: puedes encontrarla buscando "Doxywizard" en el menú de inicio de Windows o directamente en la carpeta donde tengas instalado **Doxygen**:

Paso 1: Pestaña **Wizard,** opción **Project**

- **Working directory:** Selecciona la carpeta donde tienes tu código fuente.
- **Project name:** Pon el nombre de tu práctica.
- **Source code directory:** Pulsa el botón "Select..." y elige la carpeta de tu código.
- **Destination directory:** Crea una carpeta llamada **doc** dentro de tu proyecto y selecciónala aquí.

Paso 2: Pestaña **Wizard,** opción **Mode**

- **Extraction mode:** Selecciona **"All Entities"** y marca **"Include cross-referenced source code in the output"** (para navegar del documento al código).
- **Source language:** Selecciona **"Optimize for C++ output"**.

Paso 3: Pestaña **Wizard,** opción **Output**

- **HTML:** Marca "HTML", "With navigation panel" y "With search function".
- **LaTeX: Desmarca** esta opción (no generaremos PDF/Latex con Doxygen).

Paso 4: Pestaña **Wizard,** opción **Diagrams**

- Marca la opción **"Use dot tool from the Graphviz package"**.
- Activa **todas** las casillas para generar un análisis visual completo de tu código.

Paso 5: Pestaña **Expert,** opción **Build**

- Marca la casilla **EXTRACT_PRIVATE** para incluir las variables miembro privadas de las clases.

Paso 6: Pestaña **Run (Ejecución y Guardado)**

1. Pulsa el botón **"Run doxygen"**.
2. **Visualización**: Pulsa el botón "**Show HTML output**". También puedes verlo siempre que quieras abriendo el archivo index.html que hay en tu carpeta doc/html.
3. **GUARDADO:** Antes de cerrar, ve al menú *File > Save as...* y guarda la configuración en la carpeta de tu proyecto con el nombre P1.Doxyfile.

7. ENTREGABLE

El nombre de los archivos debe seguir el formato **P1X**, donde **X** es el código asignado a tu grupo de prácticas (ej: P1A5 si tu código de grupo es el A5).

Subir al Campus Virtual (**Platea**) un archivo **P1X.zip** que contenga:

1. **Carpeta del proyecto final**, la cual deberá incluir:
   - Todos los ficheros fuente (.cpp, .h, .hpp...).
   - Fichero de configuración Doxygen: P1.Doxyfile
   - Fichero compile_commands.json
   - Carpeta .vscode con los 4 ficheros .json generados.
   - Carpeta doc/html con toda la documentación generada.
2. **Memoria PDF (P1X.pdf):**
   - **Formato:** Portada, Índice, Encabezado y Pie de página.
   - **Contenido:**
     - **Descripción del Proyecto:**
       - Breve descripción funcional del proyecto seleccionado.
       - **Origen:** Indicar si el proyecto ha sido facilitado por el profesor o si es propio del alumno. En caso de ser propio, especificar la asignatura en la que se desarrolló.
     - **Validación del Proyecto:**
       - Listado de ficheros analizados.
       - Tabla con las funciones que incumplen las reglas S1541 (Complejidad) o S138 (Líneas), indicando el valor detectado por SonarQube (ej: "Complejidad: 15").
       - Capturas de pantalla evidenciando estos avisos en VS Code.
     - **Corrección de Errores/Warnings de Compilación:**
       - Captura/s de pantalla de la pestaña **Problemas** con los errores/warnings detectados (aplicando el filtro !sonar).
       - Solución aplicada para cada uno de ellos.
     - **Diagramas:** Incluir al menos un gráfico generado por Doxygen.

Fecha límite de entrega

La documentación solicitada deberá ser entregada **antes** de la **finalización** de la **última sesión** habilitada para esta práctica (**9/03/2025**).